

码道 IDE + Clang : C 语言环境配置指南

适用对象：使用华为鸿蒙电脑、第一次接触 C 语言程序设计的本科一年级同学。
示例设备：HUAWEI MateBook Fold 非凡大师；示例工作区：`~/C_Course/c`。
目标：能在码道 IDE 中编辑 `.c` 文件，在本机终端中编译、运行并测试输入输出。

1. 先认识三个不同的工具

安装 IDE 并不等于安装了 C 语言编译器。本教程采用以下组合：

工具	作用	本教程采用
IDE	编辑、保存和管理源代码	华为码道 IDE
终端	输入命令、查看输出、向程序输入数据	系统“终端”（HiShell）
编译工具链	把 C 源代码变成本机可执行程序	Harmonybrew 的 ohos-sdk / Clang

Harmonybrew 是社区维护的鸿蒙包管理器，负责下载和安装工具。Clang 才是真正编译 C 程序的工具。码道 IDE 的应用市场名称为 CodeArts Agent。[1][3][4]

本教程的日常流程：

在码道中写 `A.c` → 保存 → 在终端用 `Clang` 编译 → 运行 `./A`

1.1 先确认电脑的操作系统

本教程面向 HarmonyOS 鸿蒙电脑。如果你的华为笔记本运行的是 Windows，请使用课程已有的 Windows 版教程。

在“设置”中查看“软件版本”。截至 2026-09-15，华为官方对 MateBook Fold 安装码道 IDE 的要求是：[1]

HarmonyOS 6.1.0.135 (SP9C00E100R12P2) 及以上版本

其他鸿蒙电脑请按文末链接 [1] 核对自己的机型及完整版本号，不要只比较“6.1”这三个字符。升级系统时按系统提示完成数据备份。

1.2 本教程的验证范围

华为已明确列出码道 IDE 对 Fold 的支持。Harmonybrew 的安装文档以 MateBook Pro 为鸿蒙 PC 参考设备，未单列 Fold 的验证结果。[1][3]

本文依据官方文档和工具链配方整理，尚未在 Fold 实机完成全流程测试。请以第 7、8 节的编译运行结果确认本机环境。图形断点调试及 C 语言插件的鸿蒙兼容性不作为本教程的完成条件。

2. 安装并打开码道 IDE

1. 使用华为账号登录鸿蒙电脑。
2. 打开系统的“应用市场”。
3. 搜索 CodeArts Agent，安装华为码道 IDE。
4. 在“设置”中搜索“软件版本”，连续点击软件版本 7 次，按提示进入并启用开发者选项；若系统要求重启，先保存其他工作。
5. 启动码道。若首次启动出现协议或登录页面，按实际界面提示完成。[1]

应用市场内要搜索的名称是 CodeArts Agent。旧版 CodeArts IDE 的文档和 Windows 版 C/C++ 插件教程不能直接作为鸿蒙版的配置依据。[1][8]

本教程使用代码编辑界面。若启动后显示 AI 对话或 Space 界面，请切换到 IDE 模式，再打开课程文件夹；界面位置以当前版本为准。[7]

3. 打开“终端” (HiShell)

后续安装命令先在系统“终端”中执行。[2]

方法一：通过应用中心打开

1. 点击桌面底部的“应用中心”，或按 鸿蒙键 + A。
2. 在应用列表中找到 终端。
3. 点击打开，看到可以输入命令的窗口即可。

方法二：通过搜索打开

1. 按 鸿蒙键 + S 打开小艺搜索。
2. 输入“终端”，打开搜索到的本机应用。[9]

3.1 先练习输入一条命令

在终端中输入下面的命令，然后按回车：

```
pwd
```

它会显示终端当前所在的文件夹。再输入：

```
ls
```

它会列出当前文件夹中的文件；空文件夹可能没有输出。看到提示符重新出现后，就可以输入下一条命令。

复制命令时只复制代码框内容。不要把终端已有的提示符、教程标题或运行结果一起复制；命令中的引号、括号和连字符都应使用英文半角字符。

4. 安装 Harmonybrew 并配置 PATH

PATH 是系统搜索命令的位置列表。配置好后，终端才能根据 `brew`、`clang` 这样的名称找到对应工具。

4.1 安装前准备

按 Harmonybrew 的鸿蒙 PC 安装说明完成以下设置：[3]

1. 在“设置 → 系统 → 开发者选项”中打开开发者选项。
2. 在“设置 → 隐私和安全 → 高级”中打开“运行来自非应用市场的扩展程序”。
3. 如果已安装 GitNext 或 DevBox，先备份其中需要保留的数据，再按项目说明卸载冲突应用；没有安装则跳过。
4. 保持网络连接。安装所需软件的命令在鸿蒙电脑的系统终端中执行。

4.2 安装包管理器

在终端复制以下整行命令，按回车；按安装器提示完成安装：

```
zsh -c "$(curl -fsSL https://harmonybrew.atomgit.com/install.sh)"
```

这条命令会从 Harmonybrew 项目站点获取并执行安装脚本。若出现下载或安装错误，先处理错误，不要继续粘贴后面的命令。

4.3 配置命令搜索路径

安装结束后，优先执行安装器最后显示的环境配置命令。默认安装路径下，项目给出的设置是：[3]

```
echo 'eval "$(/storage/Users/currentUser/.harmonybrew/bin/brew shellenv)'" >> ~/.zshrc
eval "$(/storage/Users/currentUser/.harmonybrew/bin/brew shellenv)"
```

- 第一行把配置追加到 `~/.zshrc`，供以后打开终端时加载，通常只需执行一次。
- 第二行让配置在当前终端立即生效。
- `~` 表示当前用户的主目录；`.zshrc` 是该用户的终端配置文件。

上面的第一行较长，复制时应保持为一条完整命令；不要在引号中手动插入换行。鸿蒙终端用户环境变量的配置机制也可查阅华为说明 [10]。

4.4 检查安装结果

```
brew --version
```

正确现象：显示 Homebrew 的版本信息。如果提示 `command not found: brew`，先核对安装器是否成功结束，并重新执行上面的第二行 `eval` 命令。

5. 安装并检查 C 编译器

依次执行：

```
brew update
brew install ohos-sdk
```

安装成功后检查：

```
command -v clang
clang --version
```

正确现象：第一条显示编译器入口路径，通常在 `.harmonybrew/bin` 中；第二条显示 Clang 版本信息。若找不到编译器，先排查安装和 PATH，再继续。

Harmonybrew 的 `ohos-sdk` 提供编译器及配套 SDK，并给链接器加上默认签名包装，使编译出的程序能够满足鸿蒙 PC 的签名要求。请通过包管理器提供的 `clang` 入口使用它。[4][5]

初学单文件 C 程序，安装 `ohos-sdk` 即可。以后需要 Make 等构建工具时，可以安装 `devel-base`。若安装 `llvm-gcc-compat` 后出现 `gcc` 命令，它实际仍指向 Clang，不是 GNU GCC。[4]

6. 创建并打开课程文件夹

6.1 在终端创建文件夹

```
mkdir -p ~/C_Course/c/C1
cd ~/C_Course/c
pwd
```

`mkdir -p` 创建多层目录；已经存在的文件夹会保留。`pwd` 的输出是工作区完整路径，请记住它，下一步要打开同一个文件夹。

6.2 在码道中打开它

1. 回到码道 IDE 的代码编辑界面。
2. 使用“打开文件夹”入口，选择上一步 `pwd` 显示的 `c` 文件夹。
3. 在左侧文件列表中展开 `C1`。
4. 在 `C1` 下新建文件，命名为 `A.c`。文件扩展名必须是 `.c`。

打开的是整个 `c` 文件夹，后续可以在它下面继续建立 `C2`、`C3` 等上机目录。完成第 7 节后，文件关系应为：

```
~/C_Course/c/
├── C1/
│   ├── A.c    ← 自己编辑的 C 源代码
│   └── A      ← Clang 编译后生成的可执行程序
```

本教程通过终端调用编译器，不需要复制 Windows/macOS 教程中的 `.vscode` 配置文件。

7. 编写、编译和运行第一个程序

7.1 在码道中编辑 A.c

将下面的代码写入 C1/A.c，使用“保存”命令保存文件：

```
#include <stdio.h>

int main(void)
{
    printf("Hello, C!\n");
    return 0;
}
```

- `#include <stdio.h>`：引入标准输入输出相关声明。
- `int main(void)`：程序从 `main` 函数开始执行。
- `printf` 输出文字，`\n` 表示换行。
- `return 0;`：表示程序正常结束。

7.2 在系统终端中编译

回到终端，依次执行：

```
cd ~/C_Course/c/C1
clang A.c -std=c11 -Wall -Wextra -pedantic -O0 -g -o A
```

编译成功通常没有输出，并且当前目录下会生成 `A`。如果出现 `error:`，先修改最早报告的错误并保存，然后重新编译。

参数	含义
-std=c11	按 C11 标准编译
-Wall -Wextra -pedantic	显示常见问题和标准符合性警告
-O0 -g	关闭优化，并保留调试信息
-o A	把生成的可执行程序命名为 A

7.3 运行生成的程序

```
./A
```

正确现象：终端输出 `Hello, C!`，随后重新出现命令提示符。

`./` 表示当前目录，`./A` 表示运行当前目录中的 `A`。鸿蒙版生成的程序不需要 `.exe` 后缀；`A.c` 是源代码，不能当作程序直接运行。

改完源代码后必须先保存、再重新编译。可以把输出改为 `Hello, HarmonyOS!`，重复上述操作，确认输出随修改更新。

8. 练习 scanf 输入和日常运行

8.1 用求和程序检查输入输出

把 A.c 改为以下内容，然后保存：

```
#include <stdio.h>

int main(void)
{
    int a, b;
    if (scanf("%d%d", &a, &b) != 2) {
        return 1;
    }
    printf("%d\n", a + b);
    return 0;
}
```

在系统终端执行：

```
cd ~/C_Course/c/C1
clang A.c -std=c11 -Wall -Wextra -pedantic -O0 -g -o A && ./A
```

&& 表示前面的编译成功后才运行程序，避免编译失败时误运行旧的 A。

程序会等待输入。在同一个终端中键入 2 3，按回车，应输出 5。如果程序仍在等待输入，命令提示符不会出现；需要中断运行时按 Ctrl+C。

8.2 使用固定测试输入

也可以把数据交给程序，快速重复测试：

```
printf '2 3\n' | ./A
printf '%s\n' '-4 7' | ./A
```

两次结果应分别为 5、3。这里终端命令 printf 生成输入文本，竖线 | 将它送入程序的标准输入。

8.3 以后每次做题的顺序

1. 在码道中打开课程文件夹，编辑目标 .c 文件。
2. 保存文件；在终端进入该文件所在目录。
3. 执行编译成功后再运行的命令，在终端输入测试数据。
4. 检查结果。若编辑的是 B.c，请把命令中的 A.c、-o A、./A 全部对应改为 B。

若当前码道版本提供集成终端，也可在其中使用相同命令；先执行 pwd 和 clang --version 检查目录和环境。集成终端与系统终端的环境可能不同，系统终端是本文的基准操作入口。

9. 常见问题排查

问题 1：应用市场找不到码道 IDE

搜索 CodeArts Agent。核对第 1 节的完整系统版本要求及文末机型列表 [1]。不要下载 Windows 安装包代替鸿蒙版。

问题 2：找不到 brew 或 clang

先在出错的终端中执行：

```
eval "$(/storage/Users/currentUser/.harmonybrew/bin/brew shellenv)"
command -v brew
command -v clang
clang --version
```

若 brew 可用但 clang 不存在，检查 `brew list ohos-sdk`；如果尚未安装该包，执行 `brew install ohos-sdk`。不要只因为能打开 IDE 就认为编译器已经装好。

问题 3：编译报错找不到 stdio.h

先确认调用的是 Harmonybrew 安装的 clang，而不是其他目录中的编译器；检查 ohos-sdk 是否完整安装。不要随意从网络下载 `stdio.h` 放进项目，标准头文件必须与工具链配套。

问题 4：运行 ./A 提示 Permission denied

先确认最近一次编译成功，并执行 `ls -l A` 检查文件。若没有执行权限，可对自己编译的 A 执行 `chmod u+x A` 后重试。若仍失败，核对“运行来自非应用市场的扩展程序”开关、编译器入口和工具链签名流程；签名问题不能只靠 `chmod` 解决。[3][5][6]

问题 5：程序一直没有输出，或输出仍是旧内容

使用 `scanf` 的程序可能正在等待输入。先键入测试数据并按回车。若输出仍旧，检查是否保存源文件、是否进入正确目录，以及是否编译成功；用第 8 节的 `&&` 命令避免误运行旧文件。

问题 6：代码有红色波浪线，但能编译

编辑器的语言分析与 Clang 编译是两套过程。先以终端的编译结果为准。C 语言补全插件应确认支持鸿蒙主机，不要直接安装 Windows/Linux 的原生插件组件。

问题 7：能不能像另外两份教程一样按 F5 调试？

本教程没有验证鸿蒙版码道与 C 调试插件之间的集成。仅有 `-g` 参数或 LLDB 文件，并不代表图形调试已经配置完成；不要直接套用另外两份教程的 F5、`cppdbg` 和调试器路径。若课程要求断点调试，请先与教师或助教确认本机支持情况。

10. 课前自检清单

完成后逐项检查：

- ☐ 电脑的 HarmonyOS 版本满足本机码道 IDE 安装要求。
- ☐ 已能打开码道 IDE 的代码编辑界面。
- ☐ 能通过应用中心打开系统“终端”，并执行 `pwd`。
- ☐ `brew --version` 和 `clang --version` 都能显示版本。
- ☐ 关闭并重新打开系统终端后，仍能找到 `clang`。
- ☐ 码道打开的是课程 c 文件夹，C1 中有 A.c。
- ☐ 能把 A.c 编译为 A，运行后显示 Hello, C!。
- ☐ 能运行求和示例，输入 2 3 得到 5。
- ☐ 知道每次修改后要先保存，再重新编译。

上述项目通过，表示单文件 C 程序的本地编辑、编译、运行和输入输出流程已经打通。它不代表图形断点调试也已完成配置。

参考资料与版本说明

资料核对日期：2026-09-15。软件和系统界面可能更新，安装要求以所链接的项目文档为准。蓝色标题可点击打开原始资料。

1. [华为云：鸿蒙 PC 安装码道 IDE](#)
机型、完整系统版本要求、应用市场名称和开发者选项。
2. [华为：鸿蒙电脑终端中如何查询并使用命令](#)
从应用中心打开终端；终端“帮助 → 用户手册”入口。
3. [Harmonybrew 项目：鸿蒙 PC 安装说明](#)
社区工具；参考设备、系统开关、冲突应用及安装命令。
4. [Harmonybrew：开发工具包说明](#)
`ohos-sdk`、`llvm-gcc-compat` 与 `devel-base` 的作用。
5. [Harmonybrew：ohos-sdk 安装配方源码](#)
Clang 入口包装、链接签名和 C 程序编译运行测试。
6. [华为：鸿蒙电脑终端中没有编译工具](#)
官方对缺少 Clang 和工具链签名的说明。
7. [华为云：什么是码道 IDE](#)
IDE 模式和 Space 模式。
8. [华为云：旧版 CodeArts IDE 的约束与限制](#)
9. [华为：鸿蒙电脑小艺搜索的功能介绍](#)
10. [华为：鸿蒙电脑终端中如何使用环境变量](#)